

RR/MVX Struggle with Multi-Threaded Programs



Relaxed Total Order (RTO)

Algorithm 1 Procedure to build $E_{nonrelaxed}$ and $E_{relaxed}$. I assume the recorder emits one particular linearization of some total order that extends $\rightarrow_{TO} \cup \rightarrow_{CO}$.

Require: A linearization of $\rightarrow_{TO}$ over the events in the execution: $\langle e_0 = \perp, e_1, e_2, \dots, e_n \rangle$
Require: $r : E \rightarrow R$ (resource of an event), $t : E \rightarrow T$ (thread of an event)
Ensure: $E_{relaxed} \subseteq E \times E$, $E_{nonrelaxed} \subseteq E \times E$

```

132:  $E_{relaxed} \leftarrow \emptyset$ 
133:  $E_{nonrelaxed} \leftarrow \emptyset$ 
134:  $lastAcquisition : R \rightarrow E \leftarrow \emptyset$   $\triangleright$  partial map, immediately previous acquisition
135: for  $i \leftarrow 1$  to  $n$  do
136:    $r_i \leftarrow r(e_i)$ ;  $t_i \leftarrow t(e_i)$ 
137:   if  $r_i \notin dom(lastAcquisition)$  then
138:      $E_{nonrelaxed} \leftarrow E_{nonrelaxed} \cup \{(e_{i-1}, e_i)\}$   $\triangleright$  First use of  $r_i$ 
139:   else
140:      $e_{prev} \leftarrow lastAcquisition[r_i]$ 
141:     if  $t(e_{prev}) = t_i$  then
142:        $E_{relaxed} \leftarrow E_{relaxed} \cup \{(e_{prev}, e_i)\}$   $\triangleright$  Same owner
143:     else
144:        $E_{nonrelaxed} \leftarrow E_{nonrelaxed} \cup \{(e_{prev}, e_i)\}$   $\triangleright$  Different owner
145:     end if
146:   end if
147:    $lastAcquisition[r_i] \leftarrow e_i$ 
148: end for
  
```

David Schwartz and Luis Pina, 2026. Optimizing Record/Replay through Relaxed Total Ordering and Multi-Version eXecution. July 02, 2026. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, Brussels, Belgium.

Process a list of TO events
Separate into two lists:
 1) **Relaxed** — can be dropped (unordered in replay)
 2) **Non-relaxed** — recorded inter-thread dependencies

RTO is built from:
 program order and non-relaxed edges

Theorem: $\rightarrow_{RTO} \subseteq \rightarrow_{TO}$

RTO is no stricter than TO because it only *drops* edges

Theorem: $\rightarrow_{PO} \subseteq \rightarrow_{RTO}$

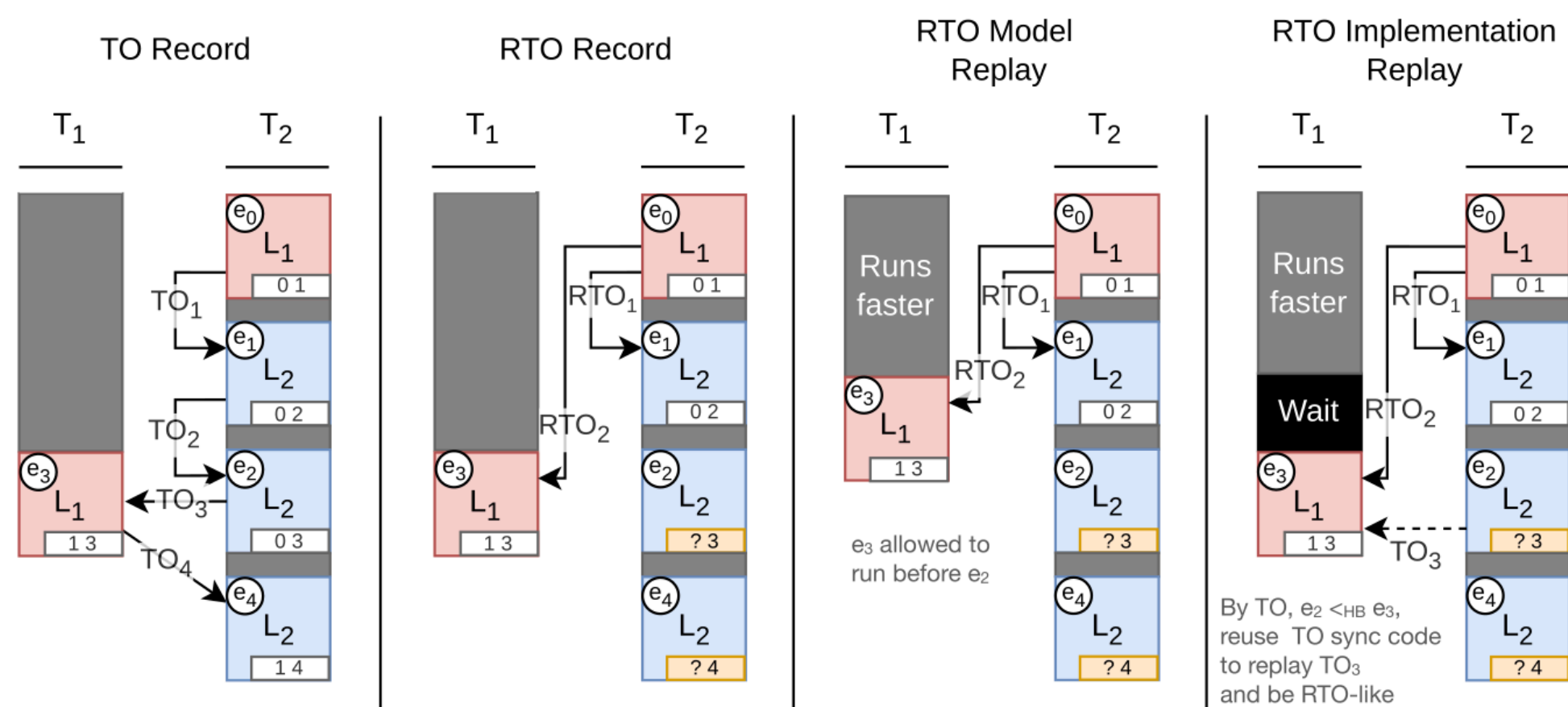
RTO is stricter than PO because first uses may order unrelated events

Formalism is in the paper!

Synchronization

Use a vector clock to order events
“blind” increments do no sync or log vector clocks

Imagine T_1 runs *faster* during replay



IO Bound Programs are Pathological

Poor performance on IO heavy programs

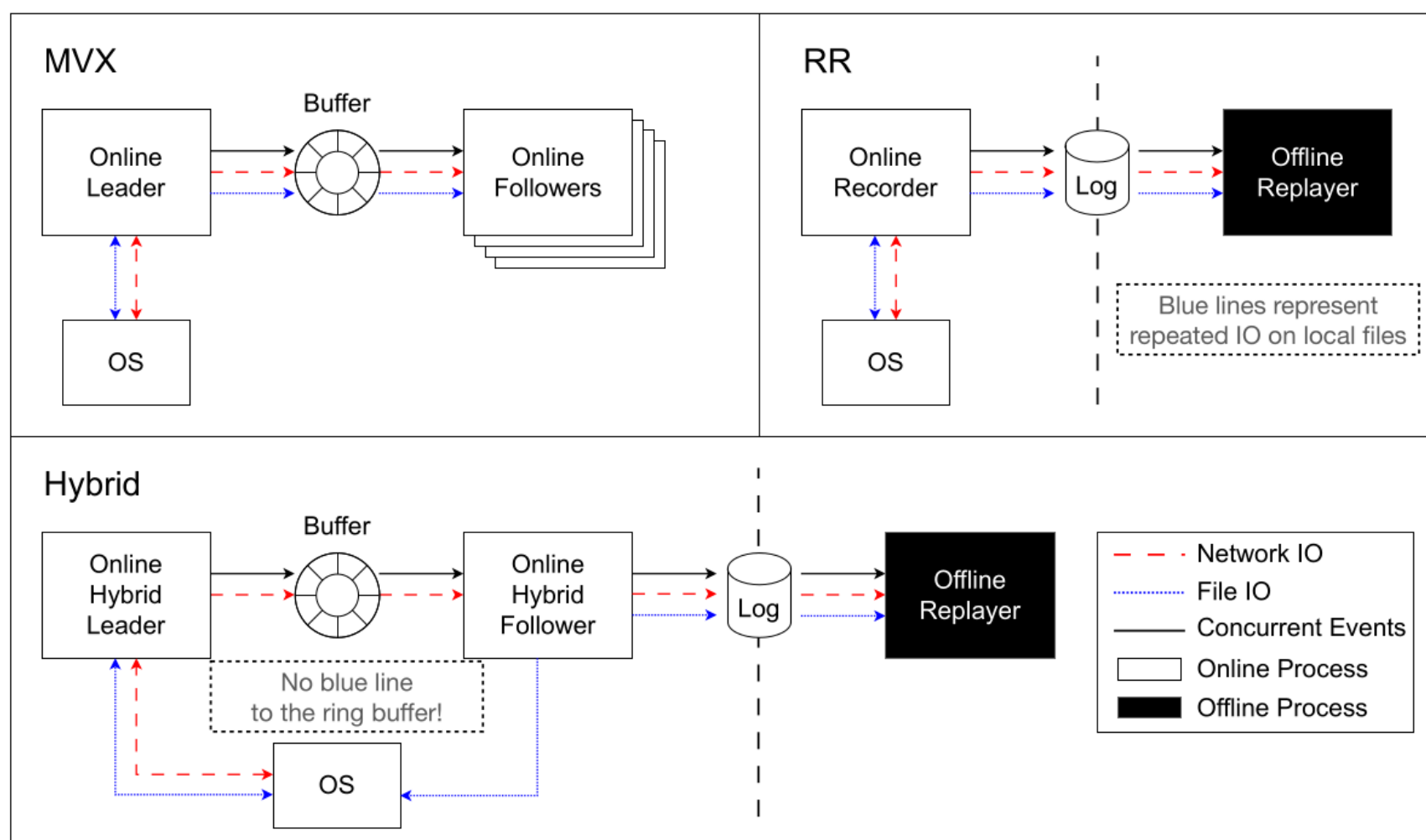
Industry tool ‘rr’ takes 6x-8x longer to record ‘dd’ copying 100MB

Why? **Duplication**

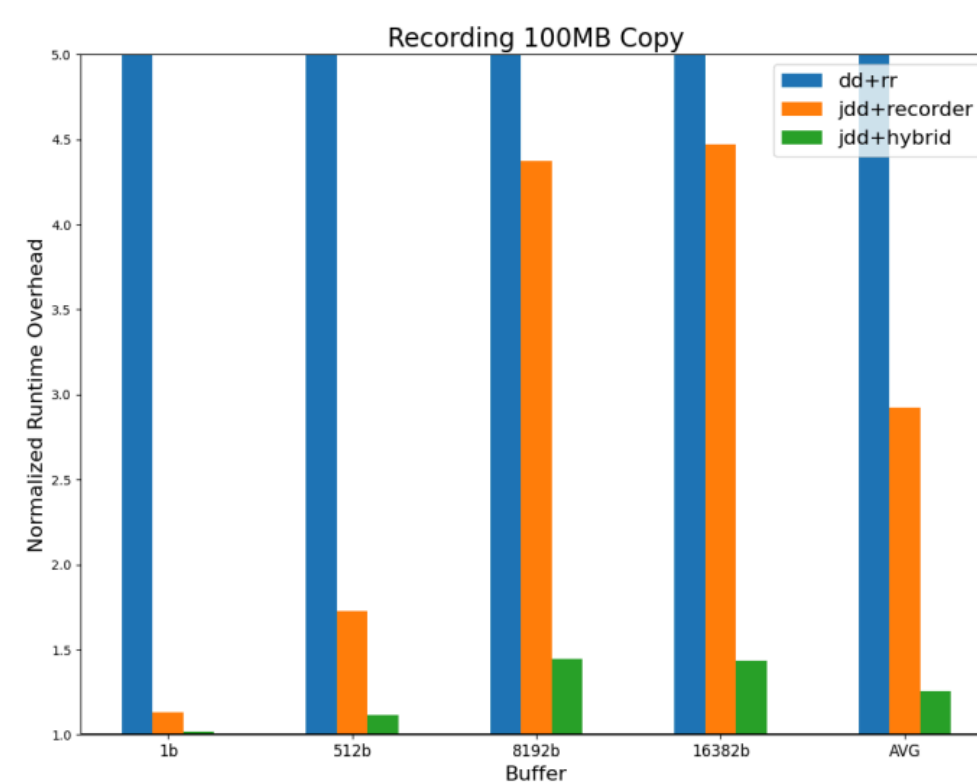
Solution:
 Recording as an MVX problem

Leader omits local file IO
 Follower **backfills** it before writing the log

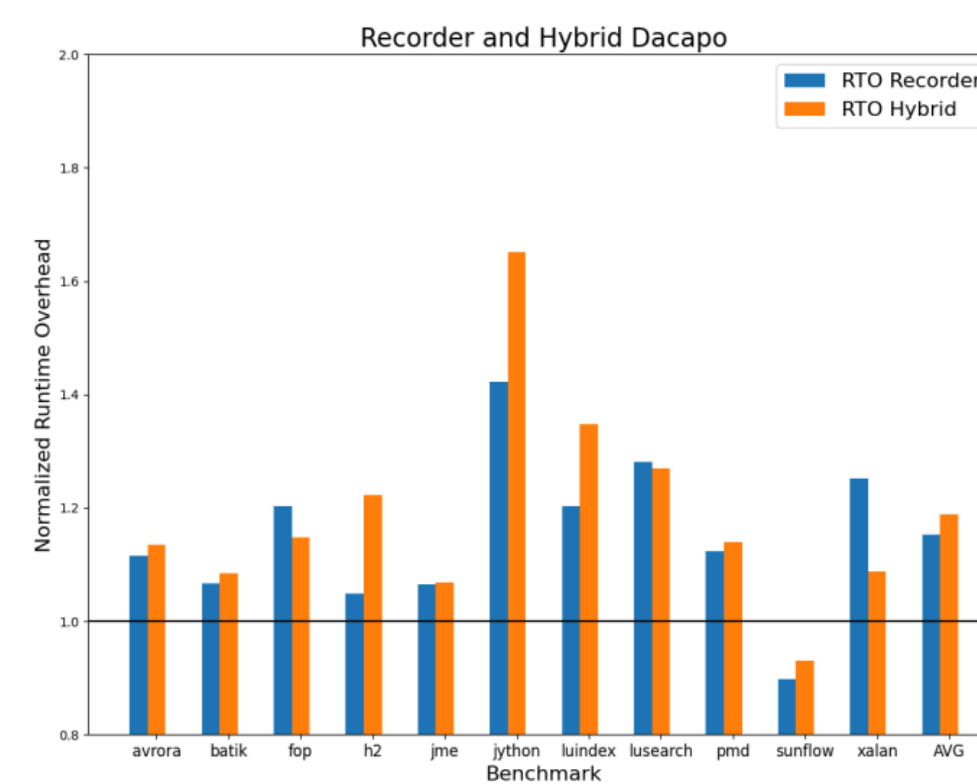
Duplication happens in the background!



Hybrid Recorder Evaluation



Massive benefits for IO bound programs!



Not making CPU bound programs worse...

Funding

This work was funded in part by NSF CCF-2227183

Acknowledgements

We would like to thank:

1. Dr. William Mansky for his review of early versions of our formal model
2. David Woloch, Linfeng, Li, Meer Shah, Luis De Santos, and Jason Pereira for their work on the Jmvx prototype
3. John Fike for IT support during development and testing
4. Many anonymous reviewers for their insightful comments

QR codes



Preprint



Artifact